Name: Omar Bin Sheik Mustafa

EE7207 Assignment 2

Central Bank Policy Sentiment Classification using FinBERT

1. Problem Statement & Data Description

1.1. Problem Statement

The objective of this project is to design and implement a Natural Language Processing (NLP) system to classify financial text into sentiment categories relevant to financial decision-making. Specifically, this project focuses on classifying financial sentences into three sentiment categories: Positive, Neutral, and Negative.

This task serves as a proxy for understanding market sentiment, which is widely used in equity trading strategies, risk assessment, and macroeconomic analysis. Understanding sentiment in financial text can be challenging, due to domain-specific language, context- dependent meanings, and subtle linguistic cues.

1.2. Data Description

The dataset used in this project is the Financial PhraseBank, a widely used benchmark dataset for financial sentiment analysis. It is taken from Hugging Face Datasets and the specific subset used for this project is "sentences_allagree", which consists of about 4800 sentences, each labelled into one of three categories: 0 representing negative sentiment, 1 representing neutral sentiment, and 2 representing positive sentiment. The dataset is loaded using the following line of code:

```
dataset = load_dataset("financial_phrasebank", "sentences_allagree",
trust_remote_code=True)
```

1.3. Data Preprocessing

Several preprocessing steps were performed to prepare the dataset for model training. First, column standardisation was performed, by renamed the relevant fields to "text" and "label" to ensure consistency throughout the pipeline. The dataset was split into training, validation, and test sets using an 80%, 10%, and 10% split respectively. Stratified sampling was applied during this process to maintain the original class distribution across all splits. Finally, tokenisation was done using the FinBERT tokeniser. Padding and truncation were applied to ensure uniform input lengths, with a maximum sequence length set to 128 tokens. The code for tokenisation is shown below.

```python
MODEL_NAME = "ProsusAI/finbert"

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

def tokenize_function(examples):
    return tokenizer(
        examples["text"],
        padding="max_length",
        truncation=True,
        max_length=128
    )
```

2. Methodology

2.1. Model Selection

Two models were chosen. First, a baseline model was chosen, consisting of a TF-IDF feature representation combined with a logistic regression classifier. Second, the main model, FinBERT, was chosen. It is a transformer-based architecture pretrained on large-scale financial text corpora, and is designed to capture domain-specific semantics which enables it to better understand the language commonly found in financial documents.

FinBERT was chosen over other traditional sequence models like LSTMs because LSTMs typically require large amounts of labelled data to effectively learn domain-specific context, which may not be feasible given the size of the dataset used in this project. Meanwhile, FinBERT leverages transfer learning, which allows it to utilise knowledge acquired during pretraining. This results in stronger understanding of financial terminology and context, such as interpreting phrases correctly within a financial setting.

2.2. Model Architecture

The baseline model follows a straightforward pipeline in which textual data is first converted into numerical features using TF-IDF vectorisation, with a maximum of 5000 features. These features are then passed into a logistic regression classifier, which performs the final sentiment classification.

The FinBERT model pipeline first tokenises the input text using a pretrained tokeniser associated with the model. The tokenised inputs are then processed by a transformer encoder, which captures contextual relationships between words. Finally, a classification head is applied to the encoded representations to produce predictions across three output classes corresponding to the labels.

2.3. Training Configuration

The following table shows the model parameters and their respective values.

Parameter	Value
Learning Rate	2e-5
Batch Size	16
Epochs	5
Weight Decay	0.01
Early Stopping	Patience = 2
Evaluation Strategy	Per epoch

A learning rate of 2e-5 was chosen because it is commonly used for fine-tuning BERT-based architectures to avoid large updates that could disrupt pretrained weights. A batch size of 16 was selected to balance computational efficiency and memory constraints. Training was conducted over 5 epochs with 0.01 weight decay to reduce overfitting. Early stopping with patience of 2 epochs was implemented, allowing training to stop if validation performance does not improve. Model evaluation was conducted per epoch to monitor performance throughout training. The code for the training setup is shown below.

```
training_args = TrainingArguments(
    output_dir="./results",
    evaluation_strategy="epoch",
    save_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=5,
    weight_decay=0.01,
    logging_dir="./logs",
    load_best_model_at_end=True,
    metric_for_best_model="f1",
    greater_is_better=True,
    report_to="none"
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    compute_metrics=compute_metrics,
    callbacks=[EarlyStoppingCallback(early_stopping_patience=2)]
)
```

2.4. Evaluation Metrics

The following metrics were used: accuracy, precision, recall, and F1-score. Macro-averaging was used for precision, recall, and F1-score to ensure that each class was treated with equal importance, regardless of class imbalance. The function written to compute the metrics is shown below.

```
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=1)

    precision, recall, f1, _ = precision_recall_fscore_support(
        labels, preds, average='macro'
    )
    acc = accuracy_score(labels, preds)

    return {
        "accuracy": acc,
        "f1": f1,
        "precision": precision,
        "recall": recall
    }
```

3. Results and Evaluation

3.1. Baseline Model

The baseline model's performance is shown in the table below.

	Precision	Recall	F1-Score	Support
0	0.86	0.61	0.72	31
1	0.88	0.96	0.92	139
2	0.75	0.70	0.73	57
Accuracy			0.85	227
Macro Average	0.83	0.76	0.79	227
Weighted Average	0.85	0.85	0.84	227

While this performance displays a reasonable benchmark, it reveals clear limitations in handling the complexity of financial text. In particular, the model struggles to interpret nuanced language and relies heavily on surface-level keyword frequency rather than deeper contextual understanding. As a result, it often fails to correctly classify sentences where sentiment is implied rather than explicitly stated.

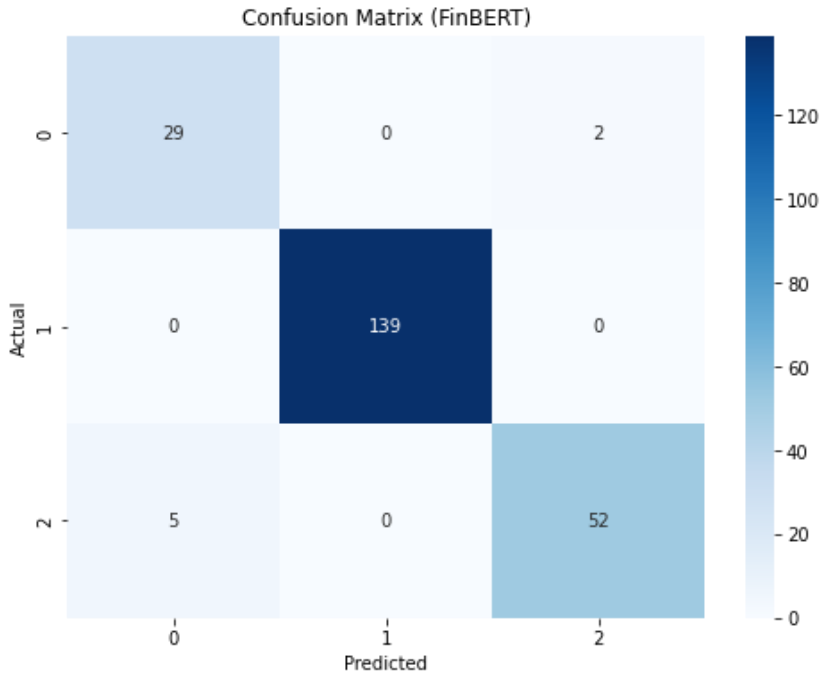
3.2. FinBERT

The FinBERT model's performance is shown in the table below.

	Precision	Recall	F1-Score	Support
0	0.85	0.94	0.89	31
1	1.00	1.00	1.00	139
2	0.96	0.91	0.94	57
Accuracy			0.97	227
Macro Average	0.94	0.95	0.94	227
Weighted Average	0.97	0.97	0.97	227

The FinBERT model demonstrates a substantial improvement over the baseline model's, achieving an accuracy of 0.97 on the test set, indicating strong predictive performance. The macro-averaged F1-score was 0.94, showing balanced performance across all classes. The weighted F1-score was 0.97, reflecting the model's effectiveness in handling the dataset's class distribution. This performance gain highlights the effectiveness of transformer-based architectures. FinBERT is better equipped to capture semantic relationships between words and interpret subtle linguistic cues, allowing it to more accurately classify financial sentiment.

An analysis of the confusion matrix, shown below, provides more insight into the model's behaviour across different classes. The model correctly classified 139 neutral samples, indicating a perfect recall for this class and highlighting its ability to recognise neutral financial language with high precision. For the negative class, 29 out of 31 instances were correctly predicted. Similarly, the positive class achieved 52 correct predictions out of 57. No neutral statements were misclassified as either positive or negative, reflecting the model's robustness in distinguishing neutral sentiment.



An analysis of the misclassified examples reveals that most errors arise from the context-dependent nature of financial language. For example, the sentence "Unit costs for flight operations fell by 6.4 percent" was labelled as positive but predicted as negative, likely due to the model associating the term "costs" with negative sentiment despite its favourable implication. These misclassifications highlight the inherent ambiguity of financial text while also indicating potential areas for improvement through larger datasets or enhanced contextual modelling.

4. Discussion

4.1. Key Findings

The results of this study highlight several important findings. Firstly, FinBERT significantly outperforms the baseline models, demonstrating the critical importance of domain-specific pretraining in financial natural language processing tasks. By leveraging knowledge acquired from large-scale datasets, FinBERT can capture nuanced linguistic patterns that traditional models fail to recognise. Secondly, the project confirms that financial text is highly context-dependent, as the same word may convey different meanings depending on its context. This complexity underscores the necessity of advanced contextual models for accurate sentiment classification. Thirdly, class imbalance within the dataset can have a noticeable impact on performance, with the neutral class dominating the distribution and consequently achieving the highest classification accuracy.

4.2. Strengths

This project presents several strengths that contribute to its effectiveness. It implements a comprehensive end-to-end pipeline that covers data preprocessing, feature engineering, model training, evaluation, and analysis. The inclusion of a baseline model enables meaningful insights such as performance comparisons and highlights the advantages of transformer-based architectures. Furthermore, the use of a transformer model, FinBERT, enhances the system's ability to interpret financial language accurately. The study also employs comprehensive evaluation metrics, ensuring a reliable assessment of model performance.

4.3. Limitations

One limitation is the relatively small size of the dataset, which may restrict the generalisation capability of deep learning models. Additionally, financial sentiment classification is inherently subjective, leading to potential ambiguity in the labels even in high-quality annotated datasets. Another limitation is the absence of temporal context, as the sentiment expressed in financial text may depend on prevailing macroeconomic conditions. Incorporating such contextual information could further enhance model performance and interpretability.

5. Conclusion

5.1. Conclusion

This project demonstrates the effectiveness of transformer-based models in financial NLP problems. The results show that FinBERT, a transformer-based model, delivers superior performance due to its domain-specific pretraining and ability to capture semantics within financial text. In contrast, traditional models struggle to interpret subtle linguistic patterns and complex economic terminology. The study also highlights the importance of proper evaluation and analysis in understanding a model's behaviour and validating its effectiveness.

5.2. Future Work

Incorporating time-series data, such as stock prices, alongside textual information would enable the integration of financial indicators, therefore enhancing predictive capabilities. Expanding the dataset with more diverse financial corpora would improve model robustness. Further comparative studies involving alternative architectures, such as RoBERTa and LSTM-based models, could provide deeper insights into a model's performance. Additionally, multimodal learning approaches that combine textual and numerical financial data might offer opportunities for more comprehensive financial analysis.

Appendix

Code (some outputs show 0% because Jupyter notebook crashed before screenshots were taken):

1. IMPORTS & REPRODUCIBILITY

```
import numpy as np
import pandas as pd
import torch
import random

from datasets import load_dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer,
    EarlyStoppingCallback
)

from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score,
    precision_recall_fscore_support,
    classification_report,
    confusion_matrix
)
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

import matplotlib.pyplot as plt
import seaborn as sns

# Set seeds
def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

set_seed(42)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

2. LOAD DATASET

```
#Financial PhraseBank
dataset = load_dataset("financial_phrasebank", "sentences_allagree", trust_remote_code=True)

df = pd.DataFrame(dataset['train'])

# Rename for consistency
df = df.rename(columns={"sentence": "text", "label": "label"})

print(df.head())
```

```
Downloading data:   0%|          | 0.00/682k [00:00<?, ?B/s]

Generating train split:   0%|          | 0/2264 [00:00<?, ? examples/s]

                                                text  label
0  According to Gran , the company has no plans t...      1
1  For the last quarter of 2010 , Componenta 's n...      2
2  In the third quarter of 2010 , net sales incre...      2
3  Operating profit rose to EUR 13.1 mn from EUR ...      2
4  Operating profit totalled EUR 21.1 mn , up fro...      2
```

3. TRAIN / VAL / TEST SPLIT

```
train_df, temp_df = train_test_split(df, test_size=0.2, stratify=df['label'], random_state=42)
val_df, test_df = train_test_split(temp_df, test_size=0.5, stratify=temp_df['label'], random_state=42)

# Convert to HF Dataset
from datasets import Dataset

train_dataset = Dataset.from_pandas(train_df.reset_index(drop=True))
val_dataset = Dataset.from_pandas(val_df.reset_index(drop=True))
test_dataset = Dataset.from_pandas(test_df.reset_index(drop=True))
```

4. BASELINE MODEL (TF-IDF + LOGISTIC REGRESSION)

```
print("\nRunning Baseline Model...")

vectorizer = TfidfVectorizer(max_features=5000)

X_train = vectorizer.fit_transform(train_df["text"])
X_test = vectorizer.transform(test_df["text"])

y_train = train_df["label"]
y_test = test_df["label"]

baseline_model = LogisticRegression(max_iter=1000)
baseline_model.fit(X_train, y_train)

y_pred_baseline = baseline_model.predict(X_test)

print("\nBaseline Results:")
print(classification_report(y_test, y_pred_baseline))
```

```
Running Baseline Model...

Baseline Results:
              precision    recall  f1-score   support

           0       0.86      0.61      0.72        31
           1       0.88      0.96      0.92       139
           2       0.75      0.70      0.73        57

    accuracy                           0.85       227
   macro avg       0.83      0.76      0.79       227
weighted avg       0.85      0.85      0.84       227
```

5. TOKENIZATION (FinBERT)

```
MODEL_NAME = "ProsusAI/finbert"

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

def tokenize_function(examples):
    return tokenizer(
        examples["text"],
        padding="max_length",
        truncation=True,
        max_length=128
    )

train_dataset = train_dataset.map(tokenize_function, batched=True)
val_dataset = val_dataset.map(tokenize_function, batched=True)
test_dataset = test_dataset.map(tokenize_function, batched=True)
```

```
tokenizer_config.json:   0%|          | 0.00/252 [00:00<?, ?B/s]
```

```
C:\Users\Admin\anaconda3\lib\site-packages\huggingface_hub\file_download.py:142: UserWarning: `huggingface_hub` cache-system uses symlinks by default to efficiently store duplicated files but your machine does not support them in C:\Users\Admin\.cache\huggingface\hub\models--ProsusAI--finbert. Caching files will still work but in a degraded version that might require more space on your disk. This warning can be disabled by setting the `HF_HUB_DISABLE_SYMLINKS_WARNING` environment variable. For more details, see https://huggingface.co/docs/huggingface_hub/how-to-cache#limitations.
To support symlinks on Windows, you either need to activate Developer Mode or to run Python as an administrator. In order to activate developer mode, see this article: https://docs.microsoft.com/en-us/windows/apps/get-started/enable-your-device-for-development
  warnings.warn(message)
```

```
config.json:   0%|          | 0.00/758 [00:00<?, ?B/s]

vocab.txt: 0.00B [00:00, ?B/s]

special_tokens_map.json:   0%|          | 0.00/112 [00:00<?, ?B/s]

Map:   0%|          | 0/1811 [00:00<?, ? examples/s]

Map:   0%|          | 0/226 [00:00<?, ? examples/s]

Map:   0%|          | 0/227 [00:00<?, ? examples/s]
```

6. MODEL INITIALIZATION

```
model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME,
    num_labels=3
)

model.to(device)
```

```
pytorch_model.bin:   0%|          | 0.00/438M [00:00<?, ?B/s]
```

```
C:\Users\Admin\anaconda3\lib\site-packages\torch\_utils.py:776: UserWarning: TypedStorage is deprecated. It will be removed in the future and UntypedStorage will be the only storage class. This should only matter to you if you are using storages directly.  To access UntypedStorage directly, use tensor.untyped_storage() instead of tensor.storage()
  return self.fget.__get__(instance, owner)()
```

```
BertForSequenceClassification(
  (bert): BertModel(
    (embeddings): BertEmbeddings(
      (word_embeddings): Embedding(30522, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (token_type_embeddings): Embedding(2, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (encoder): BertEncoder(
      (layer): ModuleList(
        (0-11): 12 x BertLayer(
          (attention): BertAttention(
            (self): BertSelfAttention(
              (query): Linear(in_features=768, out_features=768, bias=True)
              (key): Linear(in_features=768, out_features=768, bias=True)
              (value): Linear(in_features=768, out_features=768, bias=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
            (output): BertSelfOutput(
              (dense): Linear(in_features=768, out_features=768, bias=True)
              (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
          )
          (intermediate): BertIntermediate(
            (dense): Linear(in_features=768, out_features=3072, bias=True)
            (intermediate_act_fn): GELUActivation()
          )
          (output): BertOutput(
            (dense): Linear(in_features=3072, out_features=768, bias=True)
            (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
      )
    )
    (pooler): BertPooler(
      (dense): Linear(in_features=768, out_features=768, bias=True)
      (activation): Tanh()
    )
  )
  (dropout): Dropout(p=0.1, inplace=False)
  (classifier): Linear(in_features=768, out_features=3, bias=True)
)
```

7. METRICS

```
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=1)

    precision, recall, f1, _ = precision_recall_fscore_support(
        labels, preds, average='macro'
    )
    acc = accuracy_score(labels, preds)

    return {
        "accuracy": acc,
        "f1": f1,
        "precision": precision,
        "recall": recall
    }
```

8. TRAINING SETUP

```
training_args = TrainingArguments(
    output_dir="./results",
    evaluation_strategy="epoch",
    save_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=5,
    weight_decay=0.01,
    logging_dir="./logs",
    load_best_model_at_end=True,
    metric_for_best_model="f1",
    greater_is_better=True,
    report_to="none"
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    compute_metrics=compute_metrics,
    callbacks=[EarlyStoppingCallback(early_stopping_patience=2)]
)
```

9. TRAIN

```
print("\nTraining FinBERT...")
trainer.train()
```

```
Training FinBERT...
```


Epoch	Training Loss	Validation Loss	Accuracy	F1	Precision	Recall
1	No log	0.116273	0.960177	0.934660	0.924339	0.949005
2	No log	0.070896	0.986726	0.978990	0.977395	0.980643
3	No log	0.078231	0.982301	0.970707	0.966799	0.974795
4	No log	0.073481	0.986726	0.978990	0.977395	0.980643

```
TrainOutput(global_step=456, training_loss=0.17870227077551054, metrics={'train_runtime': 4188.2188, 'train_samples_per_secon
d': 2.162, 'train_steps_per_second': 0.136, 'total_flos': 476498399505408.0, 'train_loss': 0.17870227077551054, 'epoch': 4.0})
```

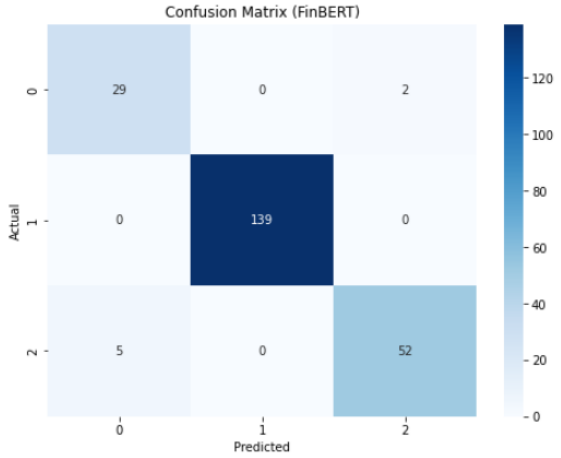
10. EVALUATE

```
print("\nEvaluating FinBERT...")
predictions = trainer.predict(test_dataset)

y_pred = np.argmax(predictions.predictions, axis=1)
y_true = predictions.label_ids

print("\nFinBERT Results:")
print(classification_report(y_true, y_pred))
```

```
Evaluating FinBERT...


FinBERT Results:
              precision    recall  f1-score   support

           0       0.85      0.94      0.89        31
           1       1.00      1.00      1.00       139
           2       0.96      0.91      0.94        57

    accuracy                           0.97       227
   macro avg       0.94      0.95      0.94       227
weighted avg       0.97      0.97      0.97       227
```

11. CONFUSION MATRIX

```
cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix (FinBERT)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



12. ERROR ANALYSIS

```
errors = test_df.copy()
errors["pred"] = y_pred

misclassified = errors[errors["label"] != errors["pred"]]

print("\nSample Misclassifications:")
print(misclassified[["text", "label", "pred"]].head(10))
```

```
Sample Misclassifications:
                                                   text  label  pred
345   Unit costs for flight operations fell by 6.4 p...      2     0
337   The new factory working model and reorganisati...      2     0
461   However , sales volumes in the food industry a...      2     0
371   Finnish power supply solutions and systems pro...      0     2
1862  In the third quarter of fiscal 2008 Efore swun...      0     2
800   Favourable currency rates also contributed to ...      2     0
1917  Cash flow from business operations totalled EU...      2     0
```